

# Lecture 5: Student Features

## CSC 1202: Software Development Project

---

Peter Wakholi (Ph.D)  
SCIT Makerere University

# Topics

- Building the Issue Submission Form
- Validating and Storing Issue Data
- Designing a Student Dashboard for Tracking Issue Status

# Student Features for AITS

- Recap of the AITS: a tool allowing students to log issues (missing marks, appeals, corrections).
- How these features fit into the overall project (user roles, workflows).
- **Objectives for Today's Session:**
  - Implement a React-based form for students to submit issues.
  - Validate and store issue data using Django (and/or DRF).
  - Create a student dashboard showing each student's logged issues and statuses.

# Learning Objectives

1. By the end of this lecture, students should be able to: Create a dynamic **React form** to capture issue details.
2. Understand **client-side vs. server-side validation** strategies.
3. Configure **Django models and views** (or DRF endpoints) to store new issues securely.
4. Build a **dashboard** in React displaying the user's issues and current status.

# Recap of System Architecture

- Where the Student Features Fit
  - Frontend: React components (forms, dashboards).
  - Backend: Django (models for Issue, endpoints to store and fetch data).
  - Database: PostgreSQL to persist issue data.
- Communication Flow:
  - User enters info in React form.
  - React sends the data to Django (API endpoint).
  - Django validates, stores, and returns a success/failure response.
  - React updates the UI (dashboard).

# Designing the Issue Submission Form

- Key Form Fields:
  - Title/Issue Subject
  - Description/Details
  - Category/Type of Issue (missing marks, appeals, corrections)
  - Course Code
  - Optional: attachments, etc.
- Form UI Best Practices:
  - Use clear labels and placeholders.
  - Provide helpful validation messages.
  - Keep it simple and user-friendly.



```
function IssueForm() {  
  const [title, setTitle] = useState("");  
  const [description, setDescription] = useState("");  
  
  const handleSubmit = (e) => {  
    e.preventDefault();  
    // Send data to backend, e.g., f  
  };  
  
  return (  
    <form onSubmit={handleSubmit}>  
      <label>Issue Title:</label>  
      <input  
        type="text"  
        value={title}  
        onChange={(e) => setTitle(e.target.value)}  
      />  
      <label>Description:</label>  
      <textarea  
        value={description}  
        onChange={(e) => setDescription(e  
      />  
      <button type="submit">Submit</button>  
    </form>  
  );  
}
```

ptual):

# Client-Side Validation in React

- **Why Client-Side Validation?**
  - Immediate feedback to the user (e.g., missing required fields, invalid input formats).
  - Reduces trips to the server for minor mistakes.
- **Common Approaches:**
  - Check for empty fields, length constraints, correct format.
  - Display error messages or disable the submit button until valid.

```
if (title.trim() === "") {  
  setError("Title is required.");  
  return;  
}
```



# Server-Side Validation & Data Storage (Django)

- Server-Side Validation ensures data integrity even if a user bypasses frontend checks.
- Django Models (e.g., Issue model with fields like title, description, student, status).
- Validation in Django can be done through model `clean()` methods or Django Forms.

```
from django.db import models
from django.contrib.auth.models import User

class Issue(models.Model):
    title = models.CharField(max_length=200)
    description = models.TextField()
    category = models.CharField(max_length=100)
    student = models.ForeignKey(User, on_delete=models.CASCADE)
    created_at = models.DateTimeField(auto_now_add=True)
    status = models.CharField(max_length=50, default='Open')
```

# Django View or DRF Endpoint

- Option 1: Traditional Django View
  - Use Django's built-in form handling (request.POST, form.is\_valid()).
- Option 2: Django REST Framework (DRF)
  - Create a Serializer for the Issue model.
  - Define a POST endpoint to handle form submissions.

# Example

## DRF Serializer (Conceptual)

```
from rest_framework import serializers
from .models import Issue

class IssueSerializer(serializers.ModelSerializer):
    class Meta:
        model = Issue
        fields = '__all__'
```

## DRF View

```
from rest_framework.views import APIView
from rest_framework.response import Response

class IssueCreateView(APIView):
    def post(self, request):
        serializer = IssueSerializer(data=request.data)
        if serializer.is_valid():
            serializer.save(student=request.user)
            return Response(serializer.data, status=201)
        return Response(serializer.errors, status=400)
```

# Connecting React Form to the Django Endpoint

---

- Security Considerations:
- Attach a CSRF token if using traditional Django views.
- Ensure authentication (e.g., JWT, session-based).

```
const handleSubmit = async (e) => {
  e.preventDefault();
  try {
    const response = await fetch("/api/issues/", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ title, description, category }),
    });
    if (response.ok) {
      // handle success
    } else {
      // handle errors
    }
  } catch (err) {
    console.error(err);
  }
}
```

# Designing the Student Dashboard

- **Key Dashboard Features:**

- List of issues logged by the currently signed-in student.
- Status indicators: Open, In Progress, Resolved, etc.
- Ability to filter or search issues (optional).

- **UI/UX Considerations:**

- Clear table or card layout to show each issue and status.
- Link to open a detailed view or track updates.

# React Example (Conceptual)

```
function StudentDashboard() {  
  const [issues, setIssues] = useState([]);  
  
  useEffect(() => {  
    fetch("/api/issues/mine/")  
      .then((res) => res.json())  
      .then((data) => setIssues(data));  
  }, []);  
  
  return (  
    <div>  
      <h2>My Issues</h2>  
      {issues.map((issue) => (  
        <div key={issue.id}>  
          <h3>{issue.title}</h3>  
          <p>Status: {issue.status}</p>  
        </div>  
      ))}  
    </div>  
  );  
}
```

# Best Practices & Common Pitfalls

- Best Practices:
  - Keep forms simple and clearly labeled.
  - Validate at multiple layers: client side + server side.
  - Provide real-time feedback to users.
  - Protect against invalid data (SQL injection, etc.).
  - Handle errors gracefully (e.g., descriptive messages).
- Common Pitfalls:
  - Missing or weak validation checks.
  - Not handling different user roles properly (e.g., a non-student user hitting the student endpoint).
  - Overly complex forms with too many fields at once.

# This Weeks Tasks

- Assignment:
  - Develop and test the issue submission feature.
  - Create a basic dashboard showing logged issues.
- Programming Task:
  - Use React to build the issue submission form.
  - Set up a Django API endpoint to receive and store issue data.